Hands-on: Unsupervised Learning

This hands-on session consists of two parts. In the first one, you will implement principal component analysis (PCA) and apply it first to clinical data and then to face images, for the purpose of dimensionality reduction. For the case of clinical data, the final objective is to distinguish between healthy and disease. In the second part, the objective is to identify pathological cardiac shapes. For that, you will need first to extract some discriminative features and then apply a clustering algorithm to them. The data and some auxiliary function (in Matlab) are available in directories data and lib, respectively.

Upload the code (either in Matlab or Python) together with the analysis of the results of all tasks to Aula Global. You can use a notebook (e.g. Matlab Live code or Jupyter) to include both code and the analysis of the results.

1 Principal Component Analysis

In this exercise, you will use PCA to perform dimensionality reduction. You will first experiment with a clinical dataset, and then use it on a larger dataset of 5000 face images.

1.1 Application to clinical data

The dataset is an adapted version of the `Chronic_Kidney_Disease` dataset from the UCI Machine Learning Repository[1]. It includes 400 patients with 24 variables (features) plus the label class (class 1 corresponds to patients with chronic kidney disease, whereas class 0 to patients without the disease). Variables are described in the file `chronic_kidney_disease_ad.info`. The dataset can be used to predict the chronic kidney disease. In this exercise, you will apply PCA to simplify the data with the aim of detecting the disease.

1.1.1 Pre-processing of the data

- Load the dataset `Chronic_Kidney_Disease.csv` (in Matlab you can use the command `csvread`, `dlmread`, or `readtable`). Why is this data a good candidate for dimensionality reduction?
- The dataset has a number of missing values (which appear as "NaN") you should deal with. One general approach to deal with missing data is to replace missing values by the attribute mean (see e.g. provided function `handle_missing`). Justify whether this approach is convenient for the `Chronic_Kidney_Disease` dataset.
- Divide the dataset into a feature set and a label vector (corresponding to the last column). Note that although you have the labels to know for each case whether it is normal or pathological, these labels are only used in this exercise for visualization purposes. Use scatter plots to visualize relationships between (some of the numerical) features (e.g. "blood urea" vs "serum creatinine"). Color the data points according to class label and see whether they are naturally clustered according to diagnosis.

[1] https://archive.ics.uci.edu/ml/datasets/

1.2 Implementing PCA

- Standardize the variables to have zero mean and unit variance. Why the latter is important?
- Implement PCA to compute the principal components. Do not use the already existing Matlab function `pca` (or any other already implemented PCA function).
- Use the principal components to reduce the feature dimension of your dataset by projecting each example onto a lower dimensional space. Visualize the first two or three dimensions of the transformed data (see e.g. Figure 1). Based on the new dimensions computed using PCA, is it easier to identify patients with chronic kidney disease?
- What is the smallest number of dimensions that contains at least 90% of the information of the data?
- After projecting the data onto the lower dimensional space, you can approximately recover the data by projecting them back onto the original high dimensional space. Compute this approximation using 10 components. Which is the reconstruction error?

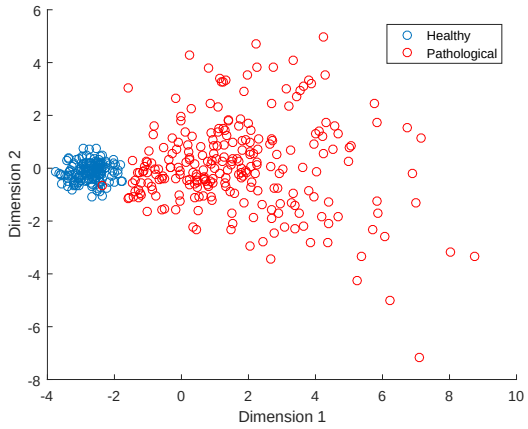


Figure 1: 2D visualization of the two first new coordinates obtained using PCA on the kidney clinical dataset.

1.3 Application to face image data

In this exercise, you will run PCA on face images for dimensionality reduction. The file `faces.mat` contains a dataset X of 5000 faces, each of size 32×32 pixels in grayscale. Each row of X corresponds to one face image (put as a vector of length 1024).

- Load the dataset `faces.mat` and visualize the first 100 faces (you can use the function `displayImageData` for it).
- Normalize the dataset to have zero mean and run PCA to obtain the principal components. Justify whether standardization (i.e. rescaling to have unit variance) is needed.
- Note that each principal component is of length 1024. You can visualize these principal components by reshaping each of them into a 32×32 matrix that corresponds to the pixels in the original dataset. Visualize the first 36 principal components that describe the largest variations (you can use function `displayImageData` for this). Observe how they capture more and more details.

- Project the face dataset onto the first 100 principal components. Now, each face image is described by a vector of length 100.
- Recover the data using the projected dataset. Visualize the first 100 reconstructed faces and compare it to the original ones.

2 Clustering: application to cardiac data

In this exercise, your objective is to distinguish (using clustering) between healthy and pathologic cardiac shapes, extracted from 2D echocardiographic data[2] at end-diastole. You are free to experiment in terms of features extraction and clustering methods, provided you can justify that your results are meaningful.

2.1 Extracting features

- Load the data in `cardiacShapes.mat`, which consists of the data from 106 acquisitions (71 volunteers and 35 cases with hypertrophic cardiomyopathy). The variables shapeData and labelT correspond to the 2D myocardial centerline of each subject and the subgroup label, respectively. The dimensions of shapeData ($106 \times 2 \times 67$) stand for the number of subjects, the x-y coordinates, and the number of points along each curve.

  Note that although you have the labels to know for each case whether it is normal or pathological, these labels should only be used to help to decide which features could distinguish healthy from disease, and to check your clustering algorithm.
- Visualize the myocardial shapes for all subjects (Figure 2, left). Can you get an intuition of some subgroup differences? Which features would you consider relevant to extract to cluster these data?
- Compute and visualize some basic features such as the curve length, the shape area (Matlab function `polyarea`), or the curvature e.g. at the apex. As an alternative to your features, you can use the ones in `cardiacFeatures.mat`.
- Plot one feature against another and check if you can have an intuition of data clusters (Figure 2, right).

2.2 Clustering

- Implement k-means (do not use already existing implementations) and use it to group the data into two clusters. Evaluate your results using the label data. Below you can find some guidelines to help you to implement k-means.

2.2.1 Implementing k-means algorithm

The k-means algorithm is a method to automatically cluster similar data examples together. Given a training set $\{\mathbf{x}_1, \ldots, \mathbf{x}_M\}$ (where $\mathbf{x} \in \mathbb{R}^D$), the objective is to group the data into a few cohesive clusters.

The algorithm is iterative: it starts by guessing the initial centroids, and it next refines this guess by repeatedly assigning examples to their closest centroids and then recomputing the centroids based on the assignments. The k-means algorithm is as follows:

[2]The data and some of the guidelines used in this part are courtesy of Prof. Nicolas Duchateau from Université Lyon, France.

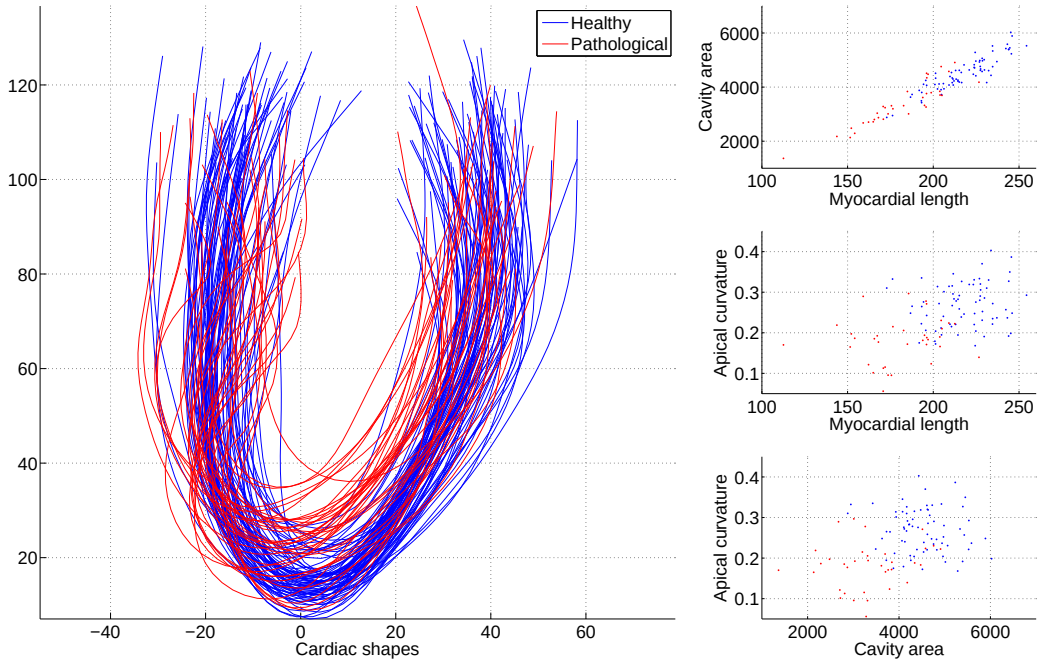


Figure 2: Myocardial shapes in the cardiac dataset and examples of extracted features.

```
% Initialize k centroids (either automatically or manually)
centroids = kMeansInitCentroids(X, k);
for iter = 1:iterations
% Cluster assignment step: assign each data point to the closest centroid.
% idx(i) corresponds to the index of the centroid assigned to data point i
idx = findClosestCentroids(X, centroids);
% Move centroid step: compute means based on centroid assignments
centroids = computeMeans(X, idx, k);
end
```

Thus, it mainly consists of two steps: (i) assigning each training example $\mathbf{x}_i$ to its closest centroid, and (ii) recomputing the mean of each centroid using the points assigned to it. Implement these two steps yourself.

The k-means algorithm will always converge to some final set of means for the centroids. However, the converged solution may not always be ideal and depends on the initial setting of the centroids. Therefore, in practice the k-means algorithm is usually run a few times with different random initializations, and the one with the lowest cost function value is chosen.